

PROGETTAZIONE E IMPLEMENTAZIONE DI UN TOOL API-AGNOSTICO PER LA SECURITY ASSURANCE DI API REST

Enea Manzi – 65935A

Relatore

Prof. Marco Anisetti

Correlatore

Prof. Claudio Agostino Ardagna

L'adozione delle architetture a microservizi cloud-native ha ridistribuito la complessità applicativa su un numero crescente di servizi autonomi comunicanti tramite API REST, moltiplicando la superficie di attacco esposta in assenza di una governance sistematica del ciclo di vita degli endpoint. Il gateway API è emerso come punto di enforcement centralizzato: concentra politiche di autenticazione, autorizzazione e rate limiting in un piano di configurazione ispezionabile e costituisce il confine tra client e servizi interni.

Le vulnerabilità più rilevanti nelle API REST sono semantiche: non emergono dall'analisi sintattica delle richieste HTTP ma richiedono la conoscenza del comportamento atteso del sistema per essere rilevate. Le prime posizioni dell'OWASP API Security Top 10 2023 sono occupate da questa classe, che include violazioni di autorizzazione a livello di oggetto, difetti di autenticazione ed esposizione non intenzionale di dati. Gli strumenti di testing automatico noti in letteratura presentano due limitazioni strutturali: scanner dinamici e fuzzer generici operano sulla sintassi delle richieste HTTP e non raggiungono queste vulnerabilità; in parallelo, stabilire automaticamente se il comportamento osservato costituisce una violazione rimane un problema aperto: la valutazione dipende dalla conoscenza del dominio del controllo specifico, e nessun approccio, sulla base della letteratura analizzata, la affronta in modo sistematico. A queste si aggiungono il compromesso portabilità-profondità e l'assenza di riproducibilità deterministica.

Questo lavoro progetta, implementa e valida *APIGuard Assurance v0.1.0*, un tool di security assurance *contract-driven* e applicazione-agnostico per API REST esposte tramite gateway, con l'obiettivo di contribuire a colmare queste lacune attraverso un approccio sistematico, deterministico e integrabile nei cicli di sviluppo continuativi.

Il lavoro si può articolare come segue:

1. **Studio dello stato dell'arte e identificazione dei gap.** È stata condotta un'analisi degli approcci esistenti al security testing delle API REST, dagli scanner dinamici agli strumenti contract-driven più recenti. I risultati mostrano che la quasi totalità dei contributi privilegia la verifica della conformità funzionale, mentre il security testing delle API REST è affrontato da una minoranza e la costruzione automatica di criteri di valutazione rimane la sfida strutturalmente meno risolta.

2. **Progettazione della metodologia.** Il tool si fonda su quattro principi: agnosticismo applicativo (tutta la conoscenza del target deriva dalla specifica OpenAPI e dal file di configurazione, senza riferimenti hardcoded); paradigma contract-driven (la specifica guida la costruzione di probe sintatticamente valide e delimita la superficie di assessment); *specified oracles* ricavati dalla conoscenza del dominio del controllo specifico; riproducibilità deterministica dell'output tra esecuzioni successive. Le verifiche sono organizzate in una tassonomia di otto domini; il box gradient mappa le precondizioni di privilegio a tre livelli di profondità di assessment; le dipendenze tra test sono dichiarate esplicitamente e risolte topologicamente da uno scheduler basato su DAG (grafo aciclico diretto).
3. **Implementazione del sistema.** La realizzazione si articola in una pipeline a sette fasi sequenziali – inizializzazione, discovery della specifica OpenAPI, costruzione dei contesti, discovery e scheduling dei test, esecuzione, teardown e generazione del report – con semantica di errore differenziata: le prime quattro sono bloccanti, le ultime tre proseguono in presenza di errori non fatali. I tool esterni sono integrati tramite connector gerarchici che separano la raccolta dei dati grezzi dalla valutazione dell'esito; il discovery dinamico consente di aggiungere verifiche senza modifiche al core; la pipeline produce exit code semantici per l'integrazione CI/CD.
4. **Validazione sperimentale.** Il tool è stato validato su Forgejo 14.0.3, esposto tramite Kong 3.9 in modalità DB-less in ambiente Docker locale. Forgejo è stato selezionato per la specifica OpenAPI generata nativamente, l'autenticazione reale con più livelli di privilegio distinti e la presenza di endpoint con diversi gradi di protezione. L'assenza di riferimenti a Forgejo nel codice ha confermato in modo pratico l'agnosticismo applicativo.

I 18 test attivi su 7 domini hanno prodotto 9 PASS, 7 FAIL, 2 SKIP e 98 finding. Il sistema ha correttamente distinto conformità e violazione in tutti i casi attesi. È stata rilevata una discrepanza effettiva tra le dichiarazioni della specifica e il comportamento del target: Forgejo dichiara globalmente il requisito di autenticazione, ma alcuni endpoint genuinamente pubblici rispondono in assenza di credenziali, dimostrando la capacità del tool di rilevare disallineamenti tra contratto e implementazione. Due run indipendenti hanno restituito risultati byte-identici, validando empiricamente il determinismo dichiarato.

Il limite strutturale principale è la dipendenza dalla qualità della specifica OpenAPI: un contratto incompleto o disallineato produce un assessment parziale senza che il sistema possa rilevarlo. Inoltre, nei deployment cloud managed l'assenza di Admin API accessibile riduce la copertura alle sole verifiche *black-box* e *grey-box*. Sul fronte degli sviluppi futuri, le priorità operative riguardano adapter per i principali gateway cloud e il completamento dei domini mancanti; sul piano della ricerca, la trasformazione in piattaforma modulare per protocolli eterogenei, la formalizzazione degli oracoli di sicurezza, la coverage augmentation progressiva e la parallelizzazione intra-batch.